

FLY FAIRLY

Engineering Take-Home Brief

The exercise: Fix Search. Design and build the end-to-end airport search flow for Fly Fairly.

Time budget: 3 to 5 hours. What you cut matters more than what you cram in.

Deliverables: Working prototype, a 1-page approach memo, and a 10 to 15 minute recorded walkthrough.

Stack: Your call. Any backend, index, library, or search approach you like.

What we are testing

You do not need travel industry knowledge. The three things that matter, roughly in this order:

- **Judgment.** Search is the foundation of our product. Can you frame a fuzzy problem, decide what to build, fake, and cut, and defend your calls?
- **LLM fluency.** How do you actually use LLMs day to day? Prompt iteration, evals, catching hallucinations. In your workflow and, where it makes sense, in the product.
- **Craft.** Senior-level code. Data modelling, ranking, state, clean abstractions, tests where they earn their keep.

The bar is judgment, not polish. We have built v1 of this ourselves, so we know what good looks like and will compare your work against ours.

The problem

Every booking on Fly Fairly starts with the user typing into a search box. If search is wrong, nothing else matters.

Airport search is deceptively hard:

- Users type IATA codes ("JFK"), city names ("Paris"), country names ("Japan"), states and provinces ("Hawaii", "Ontario"), regions and tourism aliases ("Bali", "Goa"), and typos ("Londn").
- Fly Fairly serves a global customer base. People search in their own language and script. A customer in Tokyo might type "東京", a customer in Seoul "서울", a customer in São Paulo might drop the accent and type "Sao Paulo".
- The same string can be ambiguous. "London" might mean the UK or Kentucky. "LON" is a multi-airport city code, not an airport.
- Naive fuzzy matching breaks in subtle ways. "Florida" the US state should not surface "La Florida" in Chile.
- Most airport databases are junk. They include heliports, seaplanes, military bases, and airports with no commercial service.

Researching how Kayak, Skyscanner, Booking, Trip.com, and others handle these problems is welcome and usually produces better results than reinventing from scratch. Tell us what you learned and what you borrowed.

Your freedom

You own every architectural decision. Make the calls and defend them in the memo:

- **Data.** Source your own. Public sources exist (OurAirports, IATA lists, Wikipedia, OpenStreetMap, commercial scrapes). How you ingest, clean, normalise, and prune is part of the test.
- **Search approach.** Whatever you want. A hosted service, a library, a custom index, an LLM at runtime, a hybrid. Research, pick one, justify it.
- **Ranking and disambiguation.** How you rank, how you handle typos, accents, aliases. When exact beats fuzzy. What the user sees when two airports share a name.

- **Multi-language and multi-script.** Users search in their own language. CJK, Arabic, Cyrillic, Thai, and Latin with diacritics all need to work. Cover at least English, Chinese, Japanese, and one non-Latin script of your choice.
- **UI (optional).** A React or React Native component is nice to have, not required. If you skip it, the walkthrough should show search working against your test harness.
- **Tests and eval harness.** How you prove your search beats naive substring matching, and how you would catch a regression before it ships.

Real failure cases to consider

These are real searches that have failed on Fly Fairly in production. Not a complete spec. Your memo should show how you reasoned about each class of problem.

- **"Hawaii" returns nothing.** A state or region search should surface airports in that state (HNL, OGG, KOA, LIH). Same class for "Ontario".
- **"Bali" returns nothing or the wrong thing.** Should surface Denpasar (DPS). Should not fuzzy-match "Balikpapan" (BPN).
- **"Florida" surfaces airports in Chile.** Fuzzy picks up "La Florida" across Latin America. The US state should win.
- **"Manama" returns nothing, but "BAH" works.** City-name-to-airport mapping broken. Same pattern for "Bengaluru" / BLR.
- **"TUL" returns nothing, but "Tulsa" works.** The reverse asymmetry. IATA code missing. Same pattern for "CTA" / Catania.
- **"Brussels" must work, not just "Zaventem".** Customers type the friendly city name, not the municipality name from the raw dataset.
- **"Londn" should find London.** Realistic typo tolerance. Not magical.
- **"LON" should surface a multi-airport city result** (LHR, LGW, STN, LCY, LTN). Not a specific airport.
- **"London" should disambiguate** London UK vs London, Ontario vs London, Kentucky.
- **"東京" should find Tokyo (HND, NRT).** Same shape for "北京" → Beijing, "서울" → Seoul, "دبي" → Dubai.
- **"Sao Paulo" and "São Paulo" return the same result.** Accent handling. Endonyms and exonyms too ("Roma" / Rome, "München" / Munich).

Heads up: a naive substring or fuzzy search on a single field will fail most of these. So will "index every field and hope."

Approach memo

One page. Graded as heavily as the code. Cover:

- Where your data came from and what you did to it.
- Which search approach you chose, what you evaluated, why this one.
- Which LLMs and tools you used and why (Claude, Cursor, ChatGPT, Copilot, v0, etc.).
- Your prompt iteration log. What did not work, what did, what surprised you.
- Build vs buy vs fake. Where you wrote real code, where you mocked, where you reached for a service or library.
- Where the LLM was wrong and how you caught it.
- How you would evaluate this in production. Metrics, edge cases, failure modes you actually hunted.
- What you would do differently with more time, and what you would push back on us about.

Recorded demo

10 to 15 minutes. Loom, QuickTime, OBS, whatever works. A live demo of the failure cases, a tour of your architecture, where you used LLMs, and one thing you would change with another week. Think of it as walking a senior engineer through the work on a call. We watch this before reading the code.

What we are not testing

- Travel industry knowledge. The context above is enough.
- Pixel-perfect design. Functional beats pretty.
- Backend completeness. Mocked APIs are fine where labelled.
- Feature completeness. A small, sharp thing beats a sprawling half-done thing.

How we will grade

Three axes, roughly equal weight: critical thinking and product judgment, LLM and prompting fluency, and engineering craft. A small, sharp thing with a thoughtful memo and a crisp demo beats a sprawling half-finished thing, every time.